Rapport : TP de Synchronisation pour IoT industriel

Arthur Bongiovanni - Sakun Withanage Perera

- Rapport : TP de Synchronisation pour IoT industriel
  - Prise en main du transceiver UWB
    - 2.1 Prise en main des sketchs exemples
    - 2.2 : Conception d'un premier protocole simple
  - Synchronisation MAC
    - 3.1 : Implémentation d'une synchronisation en étoile
    - 3. 2 Implémentation d'une synchronisation de SiSP
  - Ranging (synchronisation fine)
    - 4.1 Sketchs exemples
    - 4.2 Implémentation de 2M-TWR
  - Annexes

Prise en main du transceiver UWB

2.1 Prise en main des sketchs exemples

1. Identification des primitives

primitive utilisé :

- both :
  - decaduino.init() : initialise la carte
  - decaduino.engine() : process les interruptions, nécessaire pour permettre de recevoir et d'envoyer des trames
- sender :
  - decaduino.pdDataRequest(&buff, len) : envoie une trame stocké à l'addresse buff de taille len
  - decaduino.hasTxSucceeded() : indique si la dernière transmission s'est bien passé
- receiver :
  - decaduino.setRxBuffer(rxData, &rxLen) : prépare un buffer pour la reception de trames (elle sera stockée à l'addresse rxData et sa taille sera dans rxlen).
  - decaduino.plmeRxEnableRequest() : passe en mode reception
  - decaduino.rxFrameAvailable() : indique si une trame à été reçu.

2. Mise en place d'une isolation logique

Pour mettre en place une isolation logique, nous avons ajouté un tag sur les 2 premiers octets de chaque trame : txData[0] = 'a' et txData[1] = 's'.

```
// code sender
// à la création de chaque trame, on ajoute le tag au début de celle-ci
txData[0] = 'a';
txData[1] = 's';

// code receiver
// à la réception de la trame, on vérifie si le tag est présent
if(rxData[0] == 'a' && rxData[1] == 's'){
    //logic d'affichage du msg
}
```

2.2 : Conception d'un premier protocole simple

Pour implémenter un mécanisme de communication DATA+ACK avec des paramètres de timeout et de nb_retries configurables, nous avons défini l'algorithme suivant.

Du côté de l'émetteur (sender), une trame contenant les données à transmettre est envoyée. La taille de cette trame est strictement supérieure à celle du tag utilisé pour identifier l'échange. Après l'envoi, l'émetteur se place en attente de la réception d'un accusé de réception (ACK) pendant une durée maximale de timeout millisecondes.
Si aucun ACK n'est reçu à l'issue de ce délai et que le nombre de tentatives effectuées (retry) reste inférieur au paramètre nb_retries, le compteur de tentatives est incrémenté et la trame initiale est retransmise. Ce processus est répété jusqu'à la réception d'un ACK ou jusqu'à l'atteinte du nombre maximal de tentatives autorisées.

Du côté du récepteur (receiver), le système reste en attente de la réception d'une trame contenant le tag attendu. Dès qu'un message valide est reçu, un accusé de réception est immédiatement renvoyé à l'émetteur. Cet ACK est constitué d'un message minimal ne contenant que le tag associé à l'échange.

code sender :

```
cf code Decaduino_sender_2-2
```

output :

- cf annexe : **exo2_2-tx**

code receiver :

```
cf code Decaduino_receiver_2-2
```

output :

- cf annexe : **exo2_2-rx**

Synchronisation MAC

3.1 : Implémentation d'une synchronisation en étoile

1. Identification des primitives nécessaires

En complément des primitives utilisées dans les exemples sender et receiver, les fonctions suivantes sont nécessaires pour implémenter le mécanisme de synchronisation temporelle.

- Côté maître :
    - decaduino.getLastTxTimestamp() : permet de récupérer la valeur de l'horloge matérielle correspondant à la dernière transmission réussie.
    - decaduino.encodeUint64(uint64_t value, uint8_t* buffer) : permet d'encoder une valeur d'horloge sur 64 bits dans un tableau d'octets afin de l'insérer dans une trame destinée aux nœuds esclaves.
- Côté esclave :
    - decaduino.getSystemTimeCounter() : retourne la valeur courante de l'horloge locale du nœud esclave (un tick correspondant à environ 15,65 ps).
    - decaduino.getLastRxTimestamp() : permet d'obtenir le timestamp matériel associé à la dernière réception de trame.
    - decaduino.decodeUint64(uint8_t* buffer) : permet d'extraire une valeur d'horloge sur 64 bits depuis une trame reçue.

2. Conception du protocole

A. Séquence des messages et format:
Le protocole repose sur la diffusion périodique de messages clap par le nœud maître. Deux types de messages sont utilisés :

- Message de synchronisation initial
    - Format : ['a']['s']['c']
    - Rôle : déclencher une transmission afin de générer un timestamp matériel côté maître.
- Message clap contenant le timestamp
    - Format : ['a']['s']['c'][timestamp_tx]
      timestamp_tx correspond à la valeur de l'horloge du maître lors de l'envoi du message précédent.

Ces messages sont envoyés de manière périodique par le maître, avec un intervalle fixe (1 seconde dans notre implémentation).

B. Algorythmes locaux :

- Master :
    - envoie un premier message clap sans données afin de générer un timestamp matériel.
    - envoie ensuite un second message contenant le timestamp d'émission du message précédent.
    - Cette dernière étape est répétée périodiquement afin de diffuser régulièrement l'horloge du Master aux Slaves.
- Slave :
    - À la réception d'un message clap, le Slave sauvegarde le timestamp matériel de réception.
    - Lors de la réception d'un message contenant le timestamp du Master, l'esclave :
        - extrait l'horloge du Master,
        - calcule l'offset entre son horloge locale et celle du Master,

        - met à jour son offset interne afin d'aligner son horloge locale sur celle du Master.
    - À chaque message clap suivant, l'offset est recalculé afin de corriger les éventuelles dérives d'horloge.

C. Automates :
L'automate de fonctionnement peut être décrit comme suit :

- Master :
    - État Idle → attend la période d'envoi du message
    - État Transmission → Envoi du message clap vide si premier message, sinon envoie d'unmessage clap contenant le timestamp
    - Retour à l'état Idle après transmission
- Slave :
    - État Attente → Réception d'un message clap
    - État Synchronisation → Calcul et mise à jour de l'offset si un premier message 'clap' à déjà été reçu
    - Retour à l'état Attente

3. Implémentation et tests

code Master :

```
cf code DecaDuinoSender
```

output :
pas d'output pour le master dans le programme que nous avons réaliser

code Slave :

```
cf code DecaDuinoReceiverSniffer_copy
```

output :

- cf annexe : **exo3_1-slave**

Resultats expérimentaux :
On observe bien que la valeur de l'offset du slave se met à jours à la réception d'un message clap.

3. 2 Implémentation d'une synchronisation de SiSP

1. Identification des primitives nécessaires

Le protocole SiSP peut être vu comme une généralisation du protocole maître–esclave précédent, dans laquelle chaque nœud joue simultanément les rôles d'émetteur et de récepteur.
Aucune nouvelle primitive spécifique n'est donc requise par rapport à celles utilisées précédemment.

En revanche, contrairement au protocole hiérarchique, les valeurs échangées ne correspondent plus directement à l'horloge matérielle, mais à une horloge logique maintenue localement par chaque nœud. Les primitives d'encodage, de décodage et de diffusion de trames restent

identiques.

2. Conception du protocole

A. Séquence des messages et format:
cChaque nœud diffuse périodiquement un message `sisp` contenant sa valeur d'horloge logique partagée.
Le format des messages est le suivant :
`['a']['s']['S'][shared_clock]`
où :

- `['a']['s']['S']` est le tag identifiant le protocole SiSP,
- `shared_clock` correspond à la valeur de l'horloge logique partagée vers laquelle l'ensemble des nœuds doit converger.

Les messages sont envoyés en broadcast, sans notion de maître ou d'esclave.

B. Algorythmes locaux :

Chaque nœud exécute le même algorithme local :

- À l'initialisation, le nœud initialise :
  - une horloge logique locale local_clock,
  - une horloge logique partagée shared_clock, initialisée à la même valeur.
- À chaque itération de la boucle principale, les deux horloges sont incrémentées.
- Périodiquement, le nœud diffuse un message SiSP contenant la valeur courante de shared_clock.
- À la réception d'un message SiSP, le nœud extrait la valeur d'horloge reçue et met à jour sa shared_clock selon une règle de consensus simple :
  `shared_clock ← (shared_clock + shared_clock_reçue) / 2`

Ce mécanisme permet aux horloges partagées des différents nœuds de converger progressivement vers une valeur commune.

C. Automates :
Le comportement d'un nœud peut être modélisé par un automate à trois états :

- Idle : attente d'un événement (réception d'un message ou expiration de la période d'envoi),
- Réception : réception d'un message SiSP et mise à jour de shared_clock,
- Émission : diffusion périodique de la valeur courante de shared_clock.

Après chaque émission ou réception, le nœud retourne à l'état Idle.

3. Implémentation et tests

L'implémentation a été réalisée sur des nœuds identiques exécutant le même programme.

```
cf code sisp.ino
```

output :

- cf annexe : **exo3_2-sisp1**

second noeuds sisp :

- cf annexe : **exo3_2-sisp2**

Résultats expérimentaux :
Les tests montrent que la réception des messages SiSP dépend fortement de la durée de la fenêtre d'écoute. Une période d'écoute plus courte (à fréquence d'émission identique) permet de recevoir davantage de messages, mais limite néanmoins le nombre d'échanges effectifs.
Dans ces conditions, la convergence complète des horloges partagées est difficile à observer expérimentalement. Toutefois, on constate que la valeur de shared_clock évolue systématiquement dans la direction de la valeur reçue, ce qui valide le principe de consensus du protocole SiSP.

Ranging (synchronisation fine)

4.1 Sketchs exemples

- pour différencier les groupes de tp, nous avons mis sur les deux premiers octets nos initiales ('a' et 's'), ce qui implique de décaler tout les octets pour le reste du code.
- Les mesures de Two-Way Ranging montrent que la dérive des horloges dégrade significativement la précision de l'estimation de distance, même lorsque les capteurs sont immobiles. Sans correction du skew, une erreur systématique apparaît sur le temps de vol, entraînant une dispersion croissante et un biais des mesures de distance. La correction du skew permet au contraire de stabiliser le ToF et d'obtenir des estimations de distance plus cohérentes.

On peut l'observer ici (première colonne: ToF sans correction du skew, 2ème colonne: distance sans correction du skew, 3ème colonne: ToF avec correction du skew et 4ème colonne avec correction du skew)

- cf annexe : **exo4_1-twr_client**

Nous étions à environ 45-50 cm

4.2 Implémentation de 2M-TWR

1. Identification des primitives nécessaires

- Primitive d'émission d'une trame : decaduino.pdDataRequest (uint8_t * buf, uint16_t len)
- Primitive de réception de trame : decaduino.rxFrameAvailable()
- Primitive de timestamping : decaduino.getLastRxTimestamp() et decaduino.getLastTxTimestamp()
- Primitive de vérification : decaduino.hasTxSucceeded()
- Primitive pour continuer à écouter : decaduino.plmeRxEnableRequest()
- Primitive pour la gestion des transmissions différées (côté serveur pour transmettre le T3) : decaduino.alignDelayedTransmission(TRANSMITION_DELAY)

2. Format pour les messages échangés

- Pour le premier message du Client au Serveur on a txData[0]='a', txData[0]='s', txData[2] = TWR_MSG_TYPE_START.
- Pour la réponse du serveur nous avons : txData[0]='a', txData[1]='s', txData[2] =

TWR_MSG_TYPE_DATA_REPLY, t2 sur les octets 1 à 5 : decaduino.encodeUint40(t2, &txData[3]) et t3 sur les octets 6 à 10 decaduino.encodeUint40(t3, &txData[8])

3. Les algorithmes locaux

(voir le code en pièce jointe pour plus de détail)

- Serveur :

A la reception d'un message de type TWR_MSG_TYPE_START avec les bonnes identifiants, l'instant de réception est stocké dans la variable t2. Ensuite, il faut renvoyer au client un t2 et t3 (valeur estimée contrairement à TWR classique). Pour cela il faut faire une transmission différée avec decaduino.alignDelayedTransmission(TRANSMITION_DELAY). La valeur de retransmission est ici très important pour la présicion de la distance. Nous avons choisis $1000000000000/15.65 * 0.1 * 5E-13$ soit 0.1ms. Ce délai constitue un compromis entre, d'une part, la nécessité de laisser suffisamment de temps au nœud répondeur pour traiter la trame reçue et programmer une transmission fiable, et d'autre part, limiter l'impact de la dérive relative des horloges sur la précision du temps de vol.

- Client :

Le noeud client initie la conversation avec l'émission d'un message de type TWR_MSG_TYPE_START puis attends la réponse du maitre. A la reception de la réponse du maitre le temps d'arrivée de ce message est enregistrée dans t4. Ensuite on procède à l'extraction des valeurs de t2 et de t3.

Le masque 0xFFFFFFFFFF est utilisé pour contraindre les calculs de différences de timestamps à 40 bits, conformément au format du timestamp matériel du DW1000, afin de gérer correctement le overflow du compteur et éviter des erreurs lors du calcul du ToF.

En fin on calcule la distance avec la formule suivante :
tof = (((t4 - t1) & mask) - ((t3 - t2) & mask))/2;
On peut aussi ajouter un correcteur de skew comme sur l'example de TWR fourni.

4. Automate

- Client :
  - cf annexe : **automate_2M-TWR_client**
- Serveur :
  - cf annexe : **automate_2M-TWR_serveur**

5. Implémentation

(voire le code en pièce jointe)

- Résultats :
  - cf annexe : **2M-TWR_client**

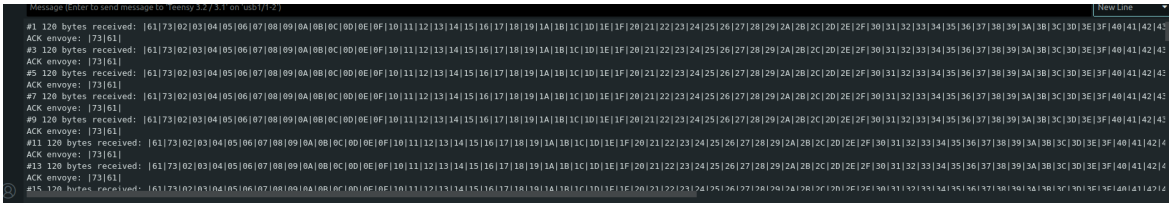
Là nous étions à environ 20cm distance entre les deux cartes. Nous avons pu observer un écart par rapport à la valeur réelle de la distace plus élevé avec le 2M-TWR qu'avec le TWR classique. Ce s'explique logiquement par l'estimation de t3 qui introduit une erreur. Cependant le TWR classique met généralement plus de temps pour faire le ranging et consomme plus d'énergie du au message supléméntaire.

Annexes

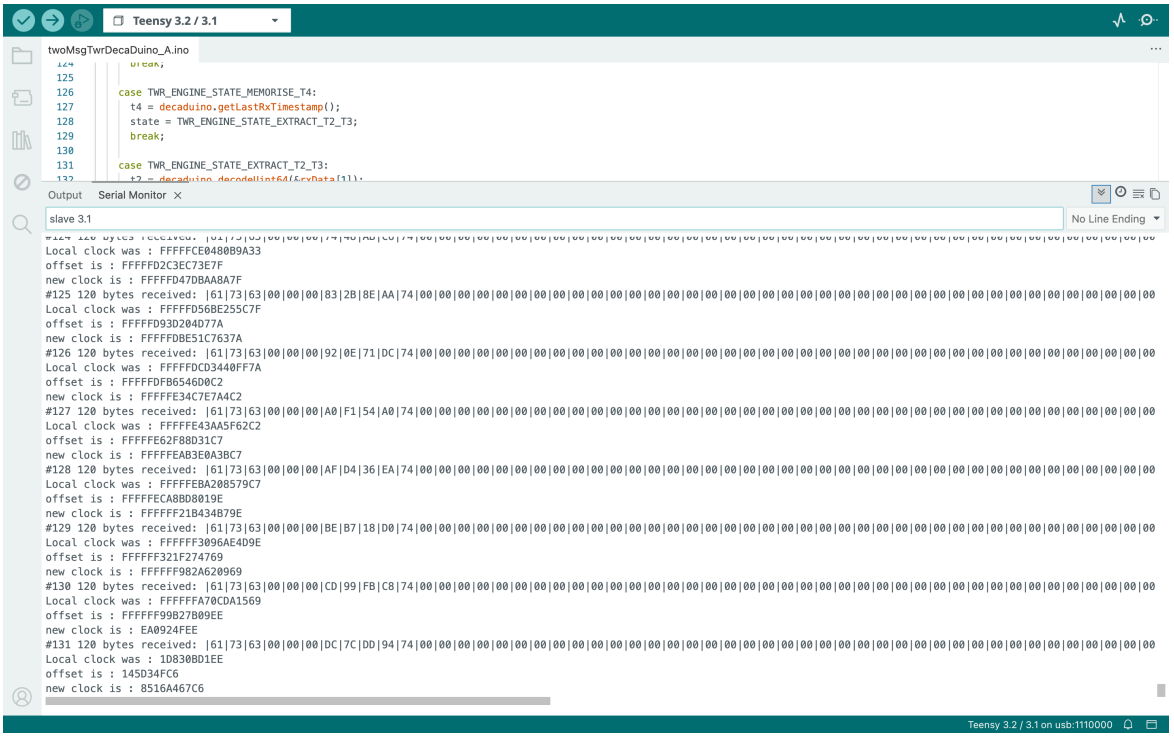
exo2_2-tx

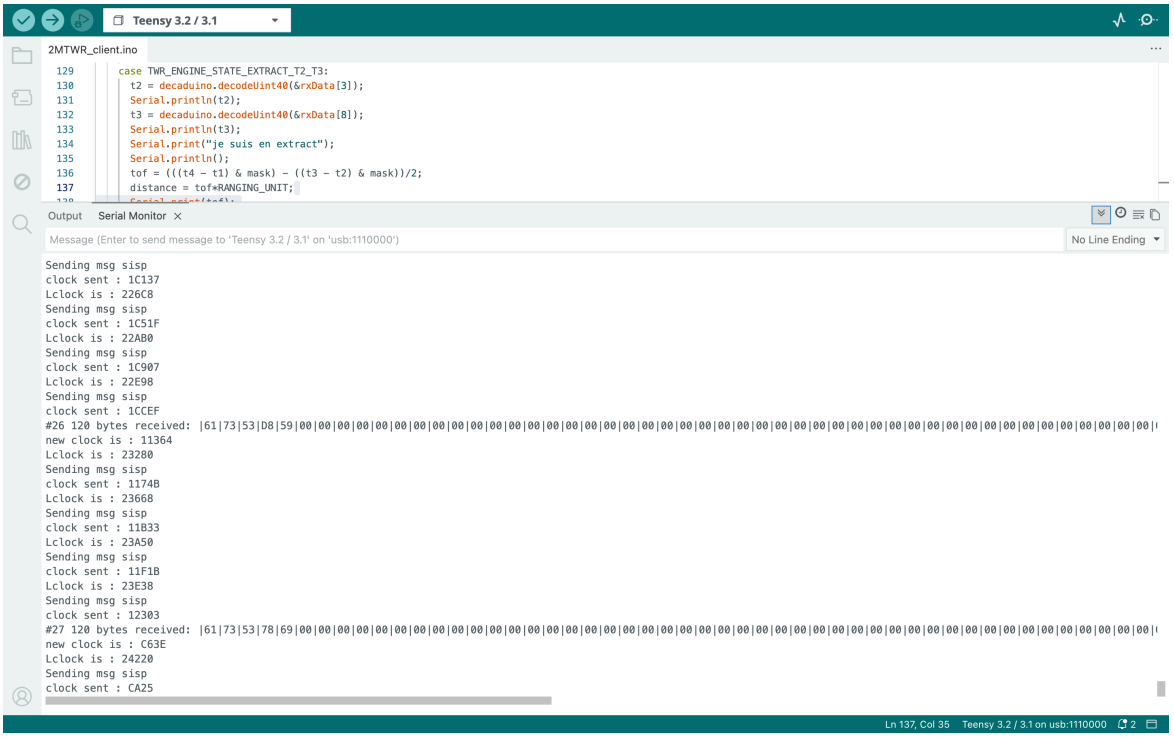


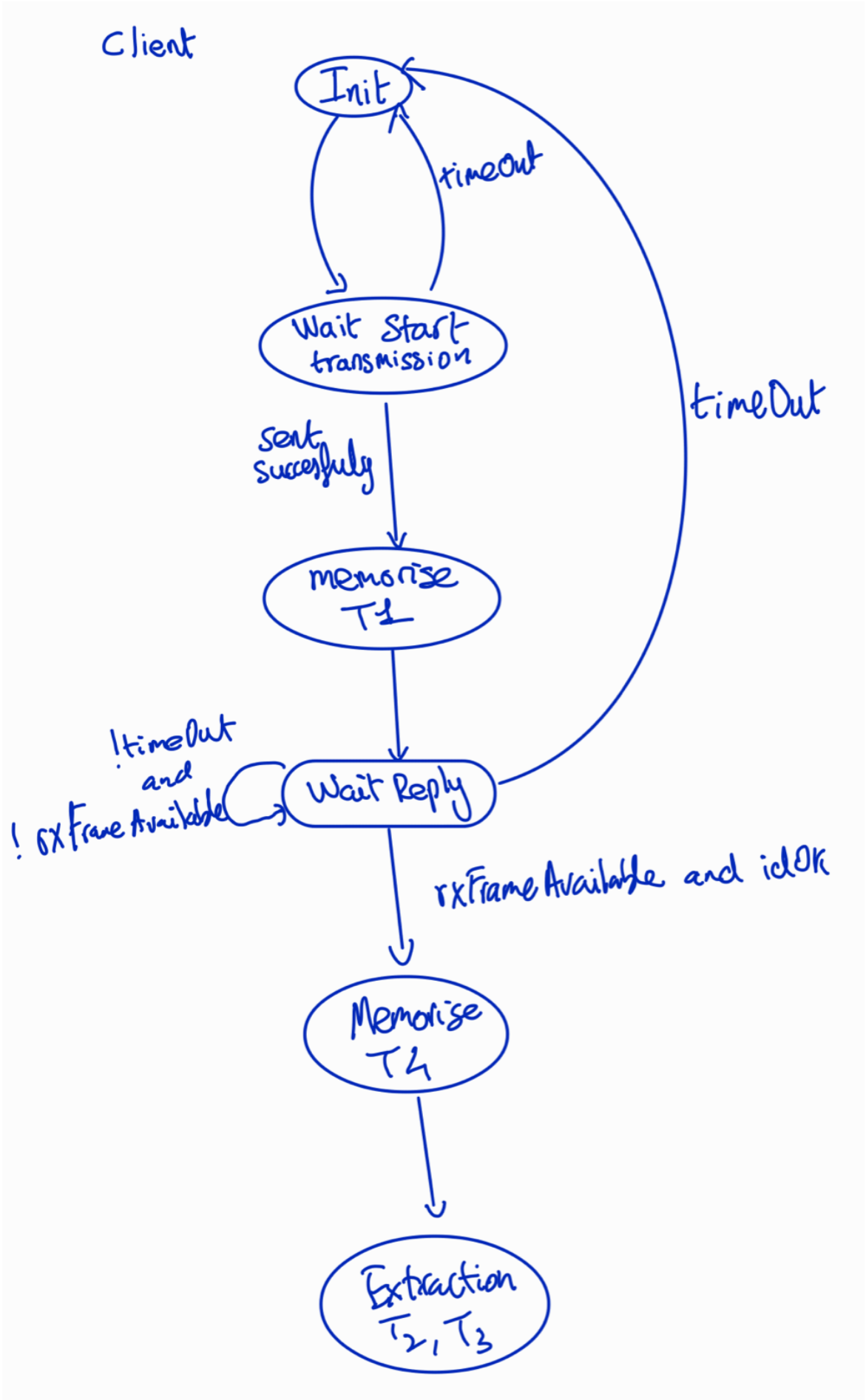
exo2_2-rx



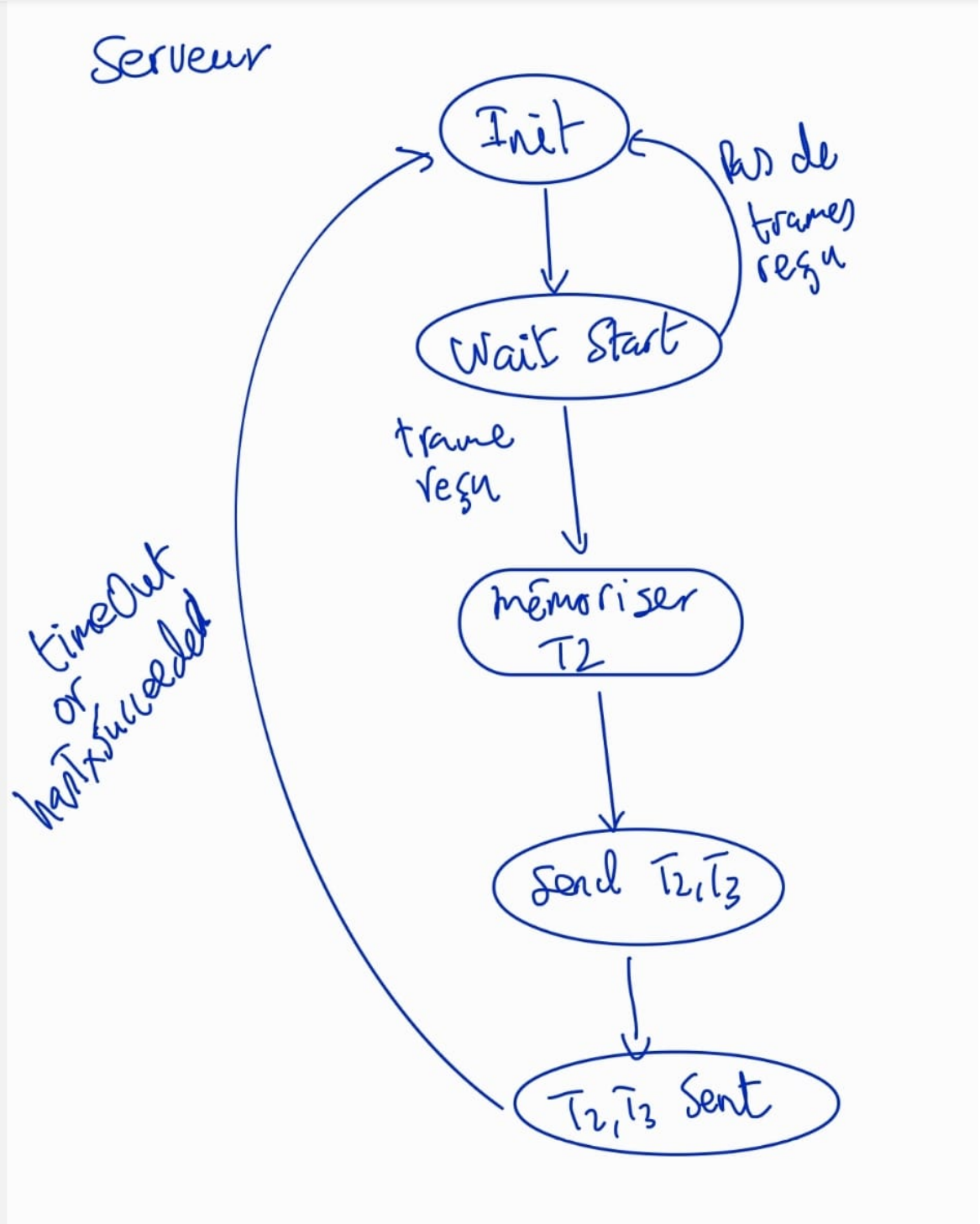
exo3_1-slave


Teensy 3.2 / 3.1
twoMsgTwrDecaDuino_A.ino
break;
case TWR_ENGINE_STATE_MEMORISE_T4:
t4 = decaduino.getLastRxTimestamp();
state = TWR_ENGINE_STATE_EXTRACT_T2_T3;
break;
case TWR_ENGINE_STATE_EXTRACT_T2_T3:
Output
Serial Monitor
slave 3.1
No Line Ending
Local clock was : FFFFFCE0480B9A33
offset is : FFFFFD2C3EC73E7F
new clock is : FFFFFD47DBAA8A7F
#125 120 bytes received: |61|73|63|00|00|00|83|2B|8E|AA|74|00|00|00

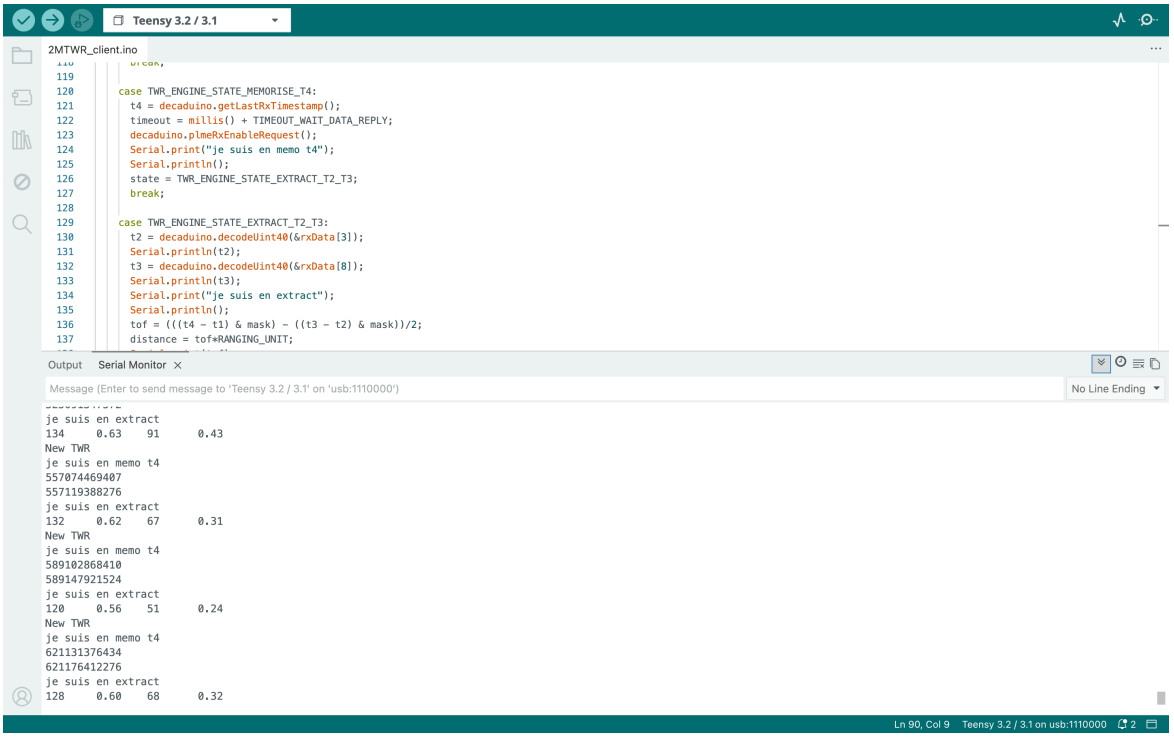

Teensy 3.2 / 3.1
2MTWR_client.ino
129 case TWR_ENGINE_STATE_EXTRACT_T2_T3:
130 t2 = decaduino.decodeUint40(&rxData[3]);
131 Serial.println(t2);
132 t3 = decaduino.decodeUint40(&rxData[8]);
133 Serial.println(t3);
134 Serial.print("je suis en extract");
135 Serial.println();
136 tof = (((t4 - t1) & mask) - ((t3 - t2) & mask))/2;
137 distance = tof*RANGING_UNIT;
Output Serial Monitor
Message (Enter to send message to 'Teensy 3.2 / 3.1' on 'usb:1110000')
No Line Ending
Sending msg sisp
clock sent : 1C137
Lclock is : 226C8
Sending msg sisp
clock sent : 1C51F
Lclock is : 22AB0
Sending msg sisp
clock sent : 1C907
Lclock is : 22E98
Sending msg sisp
clock sent : 1CCEF
#26 120 bytes received: |61|73|53|D8|59|00|00|00



automate_2M-TWR_serveur



2M-TWR_client


Teensy 3.2 / 3.1
2MTWR_client.ino
break;
case TWR_ENGINE_STATE_MEMORISE_T4:
t4 = decaduino.getLastRxTimestamp();
timeout = millis() + TIMEOUT_WAIT_DATA_REPLY;
decaduino.plmeRxEnableRequest();
Serial.print("je suis en memo t4");
Serial.println();
state = TWR_ENGINE_STATE_EXTRACT_T2_T3;
break;
case TWR_ENGINE_STATE_EXTRACT_T2_T3:
t2 = decaduino.decodeUint40(&rxData[3]);
Serial.println(t2);
t3 = decaduino.decodeUint40(&rxData[8]);
Serial.println(t3);
Serial.print("je suis en extract");
Serial.println();
tof = (((t4 - t1) & mask) - ((t3 - t2) & mask))/2;
distance = tof*RANGING_UNIT;
Output
Serial Monitor
Message (Enter to send message to 'Teensy 3.2 / 3.1' on 'usb:1110000')
No Line Ending
je suis en extract
134 0.63 91 0.43
New TWR
je suis en memo t4
557074469407
557119388276
je suis en extract
132 0.62 67 0.31
New TWR
je suis en memo t4
589102868410
589147921524
je suis en extract
120 0.56 51 0.24
New TWR
je suis en memo t4
621131376434
621176412276
je suis en extract
128 0.60 68 0.32
Ln 90, Col 9
Teensy 3.2 / 3.1 on usb:1110000